

```
1  from queue import PriorityQueue
2
3  graph = {
4      'A': [('B', 1), ('C', 3)],
5      'B': [('D', 3), ('E', 6)],
6      'C': [('F', 5)],
7      'D': [('G', 2)],
8      'E': [('G', 1)],
9      'F': [('G', 2)],
10     'G': []
11 }
12
13 heuristic = {
14     'A': 6,
15     'B': 4,
16     'C': 4,
17     'D': 2,
18     'E': 1,
19     'F': 2,
20     'G': 0
21 }
22
23 def a_star(start, goal):
24     pq = PriorityQueue()
25     pq.put((0, start))
26
27     came_from = {}
28     cost_so_far = {start: 0}
29
30     while not pq.empty():
```

Run

```
27     came_from = {}
28     cost_so_far = {start: 0}
29
30     while not pq.empty():
31         current = pq.get()[1]
32
33         if current == goal:
34             break
35
36         for neighbor, cost in
37             graph[current]:
38                 new_cost =
39                     cost_so_far[current] +
40                     cost
41
42                 if neighbor not in
43                     cost_so_far or new_cost
44                     < cost_so_far[neighbor]
45                     :
46                     cost_so_far[neighbor] =
47                         new_cost
48                     priority = new_cost +
49                         heuristic[neighbor]
50                     pq.put((priority,
51                             neighbor))
52                     came_from[neighbor] =
53                         current
54
55     path = []
```

Run

```

cost
38
39     if neighbor not in
        cost_so_far or new_cost
        < cost_so_far[neighbor]
        :
40         cost_so_far[neighbor] =
            new_cost
41         priority = new_cost +
            heuristic[neighbor]
42         pq.put((priority,
            neighbor))
43         came_from[neighbor] =
            current
44
45     path = []
46     node = goal
47     while node != start:
48         path.append(node)
49         node = came_from[node]
50     path.append(start)
51     path.reverse()
52
53     print("Shortest Path:", " -> ".join
        (path))
54     print("Total Cost:",
        cost_so_far[goal])
55
56 # Driver Code

```

Run

Shortest Path: A -> B -> D -> G

Total Cost: 6

=== Code Execution Successful ===